

Progetto B5:

Storage Chiave-Valore Replicato e Consistente in Go

Corso di “Sistemi Distribuiti e Cloud Computing” A.A. 2025/26

Docente: Valeria Cardellini
Dipartimento di Ingegneria Civile e Ingegneria Informatica
Università degli Studi di Roma Tor Vergata
cardellini@ing.uniroma2.it

Requisiti del progetto

Lo scopo del progetto è progettare, sviluppare ed orchestrare il deployment di un sistema di memorizzazione chiave-valore distribuito, replicato e fortemente consistente utilizzando il linguaggio di programmazione Go. Il sistema deve rimuovere la dipendenza da database centralizzati, implementando un meccanismo di consenso interno tra nodi.

L'architettura del sistema deve soddisfare i seguenti requisiti minimi:

- Prevedere almeno **3 tipologie di nodi distinti**, che forniscono i seguenti servizi:
 1. *Consensus node (stateful)*: un cluster di nodi che mantiene lo stato replicato e coordina la consistenza dei dati implementando gli aspetti fondamentali di un algoritmo di consenso distribuito (ad es., una versione semplificata di Paxos [1] o Raft [2]). Gestisce l'elezione del leader e la replicazione atomica dei log.
 2. *Client proxy service (stateless)*: nodo client di input, che riceve le richieste di lettura/scrittura dall'esterno, identifica dinamicamente quale nodo è l'attuale *Leader* del cluster di consenso e instrada la richiesta in modo trasparente.
 3. *Snapshot & backup service (stateless)*: servizio asincrono incaricato di interrogare i nodi di consenso per scaricare e compattare i log storici in file di checkpoint stabili.
- **Stile di comunicazione**: Le comunicazioni interne inter-servizio e di coordinamento del consenso (ad es. *AppendEntries* o elezioni in Raft) devono avvenire tramite RPC (ad es. gRPC e Protocol Buffers).
- **Pattern architetturale**: si richiede che siano integrato esplicitamente almeno 2 pattern architetturali (ad es., *Service Discovery* per la vista dinamica dei nodi di consenso, e *Circuit Breaker* per gestire il temporaneo isolamento dei servizi Client proxy o Snapshot & backup).

Si richiede di valutare la soluzione proposta includendo uno scenario di scalabilità all'aumentare del numero di nodi di consenso, misurando l'impatto sui tempi di risposta RPC, ed uno scenario di tolleranza ai guasti indotti (crash improvviso del Leader) per misurare il tempo di ri-elezione e di convergenza dello stato.

Per quanto riguarda il deployment, è richiesto l'utilizzo di **Docker Compose** per l'orchestrazione e l'esecuzione dei container su un'istanza cloud **Amazon EC2**.

Scelta e consegna del progetto

Il progetto è dimensionato per essere realizzato da **un singolo studente**. Essendo un progetto di tipo B, la valutazione peserà il 25% del voto complessivo.

La consegna del progetto deve avvenire entro il **18 settembre 2026** inviando una email alla docente che deve includere un link ad uno spazio di Cloud storage o repository (e.g., Google Drive, GitHub) contenente:

1. il codice sorgente commentato;
2. una relazione in formato PDF (massimo 5 pagine);
3. una breve guida (file `readme`) per l'installazione, la configurazione e l'esecuzione del sistema.

La relazione deve descrivere:

- l'architettura del sistema e le soluzioni adottate, motivando opportunamente le scelte progettuali;
- l'implementazione realizzata;
- la piattaforma software usata per lo sviluppo (incluse eventuali librerie di supporto) ed il testing dell'applicazione;
- risultati dei test effettuati;
- le eventuali limitazioni riscontrate.

Si consiglia di redarre la relazione in forma di articolo scientifico di lunghezza massima pari a 5 pagine, usando il formato a doppia colonna degli atti di conferenza (proceedings) ACM¹ oppure IEEE.²

Valutazione del progetto

I principali criteri di valutazione saranno:

1. rispondenza ai requisiti;
2. originalità;
3. risultati dei test;
4. organizzazione e leggibilità del codice e della relazione;
5. qualità e chiarezza della presentazione, incluse le risposte alle domande poste dalla docente.

Per ulteriori informazioni di carattere generale, fare riferimento alla presentazione dei progetti, le cui slide sono disponibili sul canale Teams del corso di Sistemi Distribuiti e Cloud Computing.

¹<https://www.acm.org/publications/proceedings-template>

²<https://www.ieee.org/conferences/publishing/templates.html>

Riferimenti bibliografici

- [1] L. Lamport. Paxos made simple. *ACM SIGACT News*, 32(4):18–25, 2001. <https://lamport.azurewebsites.net/pubs/paxos-simple.pdf>.
- [2] D. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference*, USENIX ATC'14, page 305–320, 2014. <https://www.usenix.org/system/files/conference/atc14/atc14-paper-ongaro.pdf>.